
Идея решения где **Stack**:

- 1) Храним только верхний элемент в `next` храним предыдущий
- 2) где `push` записываем в `self.head` как значение новый элемент, а в `next` старое значение `self.head`
- 3) где `pop` сократим значение `value` чтобы его вернуть и `self.head = self.head.next` т.е. теперь отслеживает предыдущее значение
- 4) где `real` просто возвращается `val` функции и где `empty` проверяем на `None`

Идея решения где **queue**:

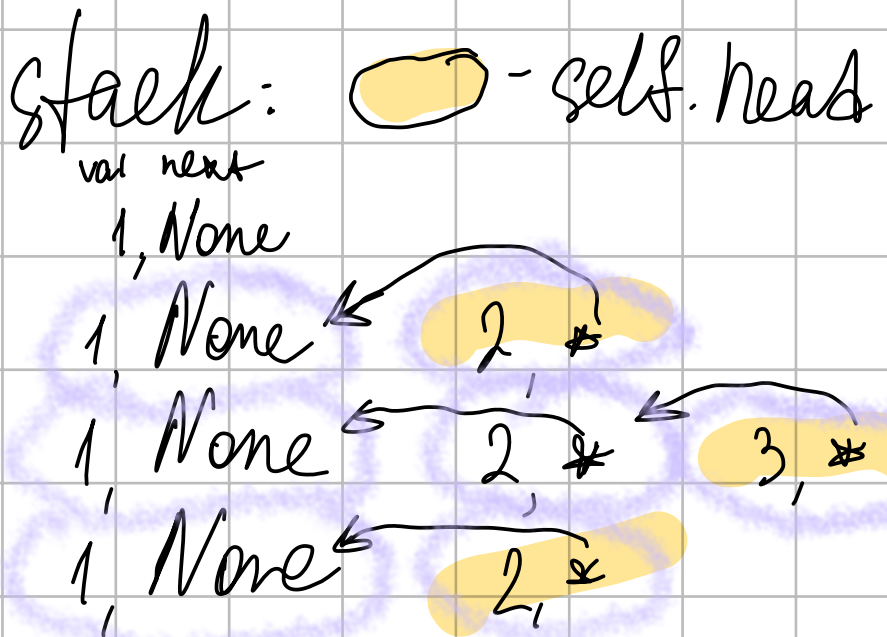
- 1) Храним первый и последний элемент в `next` храним следующий
- 2) где `enqueue` 2.1) Если это первый то записываем его в `first` и `last`

2.2) если $n \neq \text{второй}$, то записываем его в last и в first.next 2.3) если $n \neq \text{но сразу } 3+$ то записываем его в last.next и переподключаем на last

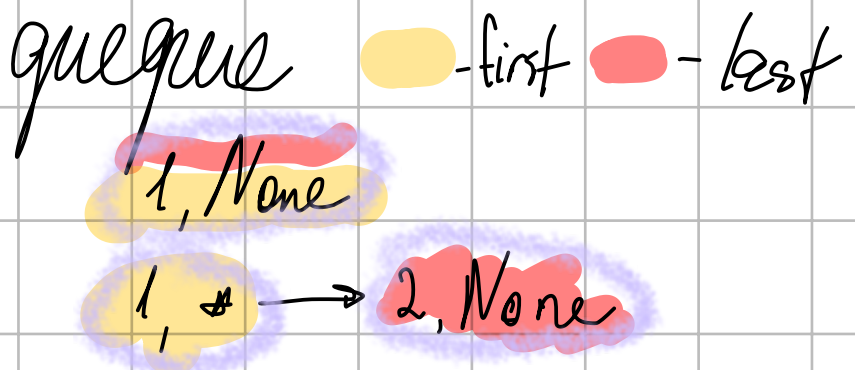
3) при dequeue записываем first.value в вывод и переподключаем $\text{first} = \text{first.next}$ (если $\text{first} = \text{None}$ то выходов не генерим)

4) при peek возвращаем first.value , при empty проверяем $\text{first} = \text{None}$

Пример qnd
push 1
push 2
push 3
pop



Пример qnd
enqueue 1
enqueue 2



enqueue 3 1, * → 2, * → 3, None
 dequeue 1, * → 2, None
 dequeue 1, * -
 dequeue None

Оценки сложности

- 1) По времени: все операции $O(1)$, т.к. не проходим по всем эл-там а работаем только с head где стек и первым и последним эл-том где очереди
- 2) По памяти: $O(n)$ потому что храним все n эл-ов, не создавая дупл. массивов

